

1. we establish a connection using mavlink with the rovr. No issue here.
 2. We create a fake goal using the Fake joystick class. No issue here.
 3. We start the imu_listener that will collect data from the rovr. No issue here.
 4. We create the Rov_gym_environment. No issue here.
 5. During environment resets, we set "self.state_history = [state] * self.history_length". This should not be an issue, it is as if the rovr was stationnary for a while.
 6. We create the agent using DeterministicGCAgent. This then creates an actor and a Critic, both based on a GRU Network. They hae the same sizes appart from inputs and outputs.
 7. If needed, we switch the goal (during exploration and first learning phase)
then, inside the step loop : {
 8. We select an action based on the phase (random or actor choice)
 - 8.1) select_action takes a state, and outputs the action, after feeding the state to the actor.
 9. Environmment step, where we apply an action for 0.1s, then compute the next state_sequence and the reward for our action.
 - 9.1) we apply the action and wait 0.1s
 - 9.2) we compute the reward for our rovr state.
 - 9.3) we convert our state after the action to a state_sequence.
 10. We convert observations and action to an appropriate format before pushing them in the replay_buffer.
 - 10.1) our obs (state_sequence) gets reshaped as : obs = np.asarray(obs, dtype=np.float32).reshape(sequence_dimension, state_dimension).
 - 10.2) we push our obs into the replay_buffer.
 - 10.2.1) all inputs are pushed as they are, without changing shape or values, and with the same priority at first.
 11. Once in every "update_every", we update either the critic only (exploration-specific) or the critic and the actor.
 - 11.1) we sample a full observation (s, a, r, s2, d) from the buffer. Here, s are still state_sequences.
 - 11.2) we calculate critic loss and actor loss according to the ddpg formulas found at "<https://spinningup.openai.com/en/latest/algorithms/ddpg.html#pseudocode>".
 - 11.3) once we updated the critic and the actor, we also update the priorities for the buffer. #This part has been commented to test out if it was a bad thing, uncomment the lign with "update_priorities" to resume it.
- }
- and we repeat 8-11 until convergence while saving occasionally.